

# Machine Learning Course Task

## ——**Bike-sharing demand prediction**

**Bokai Cai**

School of Computer Science and Technology, Soochow University



*Bokai Cai* : 20244227004

*Semester* : Winter 2024

# Abstract

本文档的主要内容分为四部分，分别是问题介绍，解决步骤（包含模型），运行结果（包含分析）和讨论。

问题介绍部分包含对问题的建模和文档要求。作者认识到需要通过过去 96h 的数据来预测未来 96h 和未来 240h 的 cnt 值。

解决步骤分为三部分，数据预处理，模型介绍以及训练和评估。数据预处理部分作者采用 pytorch 提供的 Dataset 和 DataLoader 类辅助处理。模型介绍部分作者主要使用伪代码介绍模型的原理，同时对模型性能进行理论分析。训练与评估部分作者主要介绍 run experiments 函数的内容，使用 MSE 作为损失函数，使用 MAE 和 MSE 进行评估。

运行结果包含对三种模型在短期预测和长期预测关于 MSE，MAE 和训练时间的比较，可参考表1 和表2。其中 Transformer 模型是短期预测的优选模型，N-BEATS 是长期预测的优选模型。此外，根据题目要求分别画出某一个预测未来 96h 和未来 240h 的对比图，同时绘制出采样测试集所有样本的对比图观察预测趋势。可以观察到，与真实值对比，LSTM 的预测值普遍偏大，N-BEATS 的预测值普遍偏小，Transformer 的预测值分布比较均衡。

讨论部分特别鸣谢孙游阳同学，与他的讨论使我受益匪浅。主要讨论了特征的选择以及对目标值在训练过程中的对数化处理。对于本实验而言，采取了后者来优化训练过程，同时在评估过程中对目标值进行反对数化处理来计算在原数据尺度下的 MSE 和 MAE。

此外特别说明作者对 N-BEATS 模型的改进，由于作者通过百度查询有关 N-BEATS 的时间序列预测都是单变量预测，模型本身也无法处理多特征输入。于是经过作者的思考，采用多特征拼接的方式将输入降一维，转化到单特征输入后可以很好地完成训练和评估。

最后关于模型超参数的设置，可参考表3 和表4。关于模型运行结果截图，可参考图8，图9和图10。

项目代码 `https://github.com/stu-cbk/machineLearingTask`

# Contents


**1 Introduction** **1**

**2 Procedure** **2**

2.1 Data preprocessing . . . . . 2

2.2 Model building . . . . . 3

2.2.1 LSTM . . . . . 3

2.2.2 Transformer . . . . . 4

2.2.3 N-BEATS . . . . . 5

2.3 Model training and evaluation . . . . . 6

**3 Results** **7**

3.1 Comparison and analysis . . . . . 7

3.1.1 Comparison for short-time . . . . . 7

3.1.2 Comparison for long-time . . . . . 7

3.1.3 Analysis . . . . . 7

3.2 Short-time prediction . . . . . 8

3.2.1 LSTM . . . . . 8

3.2.2 Transformer . . . . . 8

3.2.3 N-BEATS . . . . . 8

3.3 Long-time prediction . . . . . 9

3.3.1 LSTM . . . . . 9

3.3.2 Transformer . . . . . 9

3.3.3 N-BEATS . . . . . 9

**4 Discussion** **10**

4.1 Features selection . . . . . 10

4.2 Logarithmic processing . . . . . 10

**5 References** **10**

**A Hyperparameters of short-time prediction** **11**

**B Hyperparameters of long-time prediction** **11**

**C Run result screenshot** **12**

# 1 Introduction

**背景介绍：**对共享单车租赁数量进行预测可以帮助运营商合理规划和优化车辆调度，提升运营效率和服务质量。而共享单车租赁需求与环境和季节高度相关。例如，天气条件、降水、星期几、季节、一天中的时间段等都会影响租赁行为。

数据中各列的含义：

| Field | instant | dteday | season | yr | mnth | hr | holiday | weekday |
|---|---|---|---|---|---|---|---|---|
| Description | Index Number | date | Season Code | years | month | Hour | Holiday logo | Day of the week |

| Field | workingday | weathersit | temp | atemp | hum | windspeed | casual | registered | cnt |
|---|---|---|---|---|---|---|---|---|---|
| Description | Working day logo | Weather Condition Code | Actual temperature | Feeling temperature | humidity | Wind speed | Non-registered user - Vehicle | Registered User - Vehicle | Total rental quantity |

注：

season：季节编码，映射规则为：1：春季 2：夏季 3：秋季 4：冬季

yr：年份，用数字表示相对年份，例如：0：第一个年份（2011）1：第二个年份（2012）

holiday：节假日标识，为二值变量：0：非节假日 1：节假日

workingday：工作日标识，为二值变量：0：非工作日（周末或节假日）1：工作日（非周末且非节假日）

weathersit：天气情况编码，为以下类别：1：晴朗或少云 2：多云或有雾 3：雨天或雪天

casual：非注册用户租借的自行车数量。

registered：注册用户租借的自行车数量。

cnt：总租借数量，等于 casual 和 registered 的总和。

temp、atemp、hum 以及 windspeed 都为归一化值，范围为 0 到 1。

Figure 1: 数据说明

**预测任务：**根据所提供的数据对未来单车租赁数量（cnt）进行预测。基于过去 I=96 小时的数据曲线来预测未来（i）O=96 小时（短期预测）和（ii）O=240 小时（长期预测）两种长度的变化曲线（需要分别训练，即长期预测的模型参数不能用于短期预测）按照方法分为三部分。

前两部分为基础题，第三部分为开放题，各占总分的三分之一：

1. 使用 LSTM 模型进行预测；
2. 使用 Transformer 模型进行预测；
3. 使用自己提出的改进模型进行预测，结构不限。此部分以原理的新颖程度为首要评价标准，性能为次要评价标准。

**任务要求：**数据集主要分为 train 和 test 两部分。请使用两种评价标准进行测试，即均方误差（MSE）与平均绝对误（MAE）。至少进行五轮实验，并对结果取平均值，同时提供标准差（std）以评估结果的稳定性。
**提交方式：**实验报告应由以下四部分构成：1. 问题介绍、2. 模型（可以包含少量伪代码）、3. 结果与分析、4. 讨论。同时，需要提交代码（可以给出 Github 链接）。结果需以截图形式贴在报告中，并绘制出单车租赁数量（cnt）预测与真实值（Ground Truth）曲线的对比图。请注意对三种方法进行比较。

## 2 Procedure

### 2.1 Data preprocessing

下面通过伪代码描述时间序列数据构造的过程，可参考该算法手动构造数据集，也可以直接采用 pytorch 提供的 Dataset, DataLoader 类进行数据集的构造。

**Algorithm 1** Create_dataloader function

**Require:** data, seq_len, future_len, batch_size
**Ensure:** Features $X$, Targets $y$

Remove rows with missing values from $data$
Initialize empty lists $X$ and $y$
**for** i = seq_len to len(data) - future_len + 1 **do**
    Append data[i - seq_len : i, :-1] to $X$  May need to delete some useless features
    Append data[i : i + future_len, -1] to $y$  the last column is target feature "cnt"
**end for**
Convert $X$ and $y$ to numpy arrays
Initialize empty list $dataloader$
**for** start = 0 to len(X) in steps of batch_size **do**
    end = min(start + batch_size, len(X))
    Append (X[start:end], y[start:end]) to $dataloader$
**end for**
Return $dataloader$

作者采用第二种方法，数据先通过 MyDataset 类进行预处理，生成可用的特征矩阵和目标值；处理后的数据进一步被封装到 ShortDataset 或 LongDataset 中，根据不同的预测目标长度生成样本对；最终，通过数据加载器函数生成 PyTorch 的 DataLoader，便于模型进行批量训练。在上述过程中，用于短期训练的 batch_size 为 32，用于长期训练的 batch_size 为 16，提取的特征是去除"instant","dteday","cnt" 这三列后剩余的 14 列特征，预测目标值为"cnt" 列。

## 2.2 Model building

### 2.2.1 LSTM

LSTM（Long Short-Term Memory）是一种特殊的循环神经网络（RNN），主要用于处理和预测时间序列数据，尤其适用于处理具有长期依赖关系的序列数据。传统的 RNN 在处理长序列时会面临梯度消失或梯度爆炸问题，而 LSTM 通过引入特殊的门控机制有效地缓解了这个问题。

**Algorithm 2** LSTM 时间序列预测

**输入:** 数据加载器 $data_loader$，提供时间序列样本对 $\{X_{seq}, y_{seq}\}$
**输出:** 预测值 $y_{pred}$
**初始化:**
设定超参数：学习率 $lr$，训练轮次 $epochs$，输入长度 $input_size$，预测长度 $output_size$，隐藏层单元数 $hidden_nums$
初始化 LSTM 网络参数：权重矩阵 $W$ 和偏置项 $b$
初始化优化器和损失函数，例如 Adam 和 MSE
初始化 LSTM 网络的隐状态 $\mathrm{h}_0$ 和记忆单元 $\mathrm{c}_0$
**for** $epoch = 1$ **to** $epochs$ **do**
    **for** 批次 $\{X_{seq}, y_{seq}\}$ **in** $data_loader$ **do**
        重置隐状态 $\mathrm{h}_0, \mathrm{c}_0$
        **for** $t = 1$ **to** $seq_len$ **do**
            从 $X_{seq}$ 获取当前时间步的输入 $x_t$
            更新输入门、遗忘门、输出门
            更新记忆单元 $\mathrm{c}_t$
            更新隐状态 $\mathrm{h}_t$
        **end for**
        计算预测值 $y_{pred} = W_{out}\ \mathrm{h}_{seq} + b_{out}$
        计算损失 $L = MSE(y_{seq}, y_{pred})$
        反向传播计算梯度并更新参数 $W, b$
    **end for**
**end for**

尽管 LSTM 在处理长时间序列数据时优于 RNN，但它仍存在一些缺陷。首先，它对极长时间依赖的建模能力有限，难以捕获远距离时间步间的关系。其次，LSTM 的计算和训练成本较高，由于门控机制复杂且无法并行计算，训练效率相对较低。第三，在长期预测任务中，递归预测容易导致误差累积，影响精度。这些缺陷使得 LSTM 在某些场景中表现不如现代模型（如 Transformer）。

### 2.2.2 Transformer

Transformer 通过引入自注意力机制和完全并行化的处理方式，克服了 LSTM 的一些限制。与 LSTM 依赖于时间步递归的结构不同，Transformer 使用自注意力（Self-Attention）机制来直接捕获输入序列中各个位置之间的关系，不论它们之间的距离有多远。这样，Transformer 能够更有效地处理长序列，避免了梯度消失和梯度爆炸的问题。其次，由于 Transformer 结构不依赖于时间步的递归计算，Transformer 可以在训练过程中对整个序列进行并行处理，从而大大提高了训练效率。这使得 Transformer 在大规模数据集和长时间序列任务中尤为有效。

**Algorithm 3** Transformer 时间序列预测

**输入:** 数据加载器 $data_loader$，提供时间序列样本对 $\{X_{seq}, y_{seq}\}$
**输出:** 预测值 $y_{pred}$
**初始化:**
设定超参数：学习率 $lr$, 训练轮次 $epochs$, 输入长度 $input_size$, 预测长度 $output_size$, 隐藏层维度 $hidden_nums$, 自注意力头数 $heads$, 编码器/解码器层数 $num_layers$
初始化 Transformer 模型参数：编码器和解码器权重 $W_{\text{enc}}, W_{\text{dec}}$, 偏置项 $b_{\text{enc}}, b_{\text{dec}}$
初始化优化器和损失函数，例如 Adam 和 MSE
初始化位置编码 $PE$，用于输入数据的位置表示
**for** $epoch = 1$ **to** $epochs$ **do**
    **for** 批次 $\{X_{seq}, y_{seq}\}$ **in** $data_loader$ **do**
        对输入序列 $\mathrm{X}_{seq}$ 加入位置编码：$\mathrm{X}_{pos} = X_{seq} + PE$
        输入编码器：
            计算编码器输出：$\mathrm{H}_{\text{enc}} = \text{Encoder}(\mathrm{X}_{pos})$
        输入解码器：
            计算解码器输出：$\mathrm{H}_{\text{dec}} = \text{Decoder}(\mathrm{H}_{\text{enc}})$
        计算预测值：$y_{pred} = W_{\text{out}}\ \mathrm{H}_{\text{dec}} + b_{\text{out}}$
        计算损失：$L = MSE(y_{seq}, y_{pred})$
        反向传播计算梯度并更新参数 $W_{\text{enc}}, W_{\text{dec}}, b_{\text{enc}}, b_{\text{dec}}, W_{\text{out}}, b_{\text{out}}$
    **end for**
**end for**

尽管 Transformer 在时间序列预测中表现优异，但它存在计算复杂度高、内存消耗大以及对局部时间模式捕捉较弱的缺陷。自注意力机制的 $O(n^2)$ 复杂度对长序列尤为挑战。为解决这些问题，N-BEATS 模型应运而生。N-BEATS 通过简化结构，专注于时间序列的局部模式建模，避免了 Transformer 的计算瓶颈，同时无需依赖复杂的网络设计，更加高效且适用于各种时间序列预测任务。

### 2.2.3 N-BEATS

N-BEATS（Neural Basis Expansion Analysis）是一种专为时间序列预测设计的深度学习模型，旨在解决传统模型的复杂性和局部特征捕捉问题。N-BEATS 通过简单的结构，采用两个主要模块：一个是趋势模块，用于捕捉序列的长期趋势；另一个是季节性模块，专注于捕捉周期性和局部模式。N-BEATS 的最大优势在于其无需依赖特定的先验知识或复杂的网络架构，可以灵活应用于各种时间序列预测任务。通过使用全连接层和堆叠设计，N-BEATS 在训练时能有效提取时间序列的隐含特征，同时避免了计算瓶颈，提升了效率。

**Algorithm 4** N-BEATS 时间序列预测

**输入:** 数据加载器 $data_loader$，提供时间序列样本对 $\{X_{seq}, y_{seq}\}$
**输出:** 预测值 $y_{pred}$
**初始化:**
设定超参数：学习率 $lr$，训练轮次 $epochs$，输入长度 $input_size$，隐藏层维度 $hidden_size$，预测长度 $output_size$，块数 $num_blocks$，块类型 $block_type$（"trend" 或 "seasonality"）
初始化 N-BEATS 模型参数：块权重矩阵 $W_{block}$，偏置项 $b_{block}$，回归和分类分支的参数
初始化优化器和损失函数，例如 Adam 和 MSE
**for** $epoch = 1$ **to** $epochs$ **do**
    **for** 批次 $\{X_{seq}, y_{seq}\}$ **in** $data_loader$ **do**
        重置隐状态和记忆
        获取当前批次的输入序列 $X_{seq}$ 和目标值 $y_{seq}$
        **for** $block = 1$ **to** $num_blocks$ **do**
            将输入序列 $X_{seq}$ 中的多特征进行拼接处理
            将输入序列 $X_{seq}$ 传入对应的 N-BEATS 块
            根据块类型（'trend' 或'seasonality'）生成对应的隐藏层输出
            通过前馈神经网络层计算块的输出
        **end for**
        将各块的输出组合成最终的预测结果 $y_{pred}$
        计算损失 $L = MSE(y_{seq}, y_{pred})$
        反向传播并更新参数：$W_{block}, b_{block}$
    **end for**
**end for**

在实验中采用季节性模块，同时对于每一个块，将输入序列中多特征拼接起来输入，目前在该模型上大多数都是单特征预测，因此作者没有找到关于如何处理多特征数据的参考，多特征拼接算是作者的一个小技巧。

### 2.3 Model training and evaluation

下列算法用于运行多个实验并评估模型的表现。在每次实验中，模型被初始化并通过训练数据训练。训练过程中，使用给定的学习率和训练轮次更新模型。每次训练后，算法通过测试数据评估模型，计算出当前的 MSE 和 MAE，并将其存入列表中。若当前实验的 MSE 优于之前记录的最优结果，更新最优模型及其相关预测值和真实值。最终，算法计算 MSE 和 MAE 在多次实验中的平均值和标准差，并返回最优模型及其预测结果，以及 MSE 和 MAE 的平均值和标准差。

**Algorithm 5** Run Experiments with Model Evaluation

**function** RUNEXPERIMENTS(model, train_loader, test_loader, epochs, lr, nums)
    Initialize best_model, best_mse, best_predictions, best_true_values
    Initialize list for mse_list, mae_list
    **for** experiment = 1 **to** nums **do**
        Initialize model
        Call `train_model(model, train_loader, epochs, lr)`
        Call `evaluate_model(model, test_loader)`
        Update mse, mae for the current experiment
        Append current mse and mae to mse_list, mae_list
        **if** current mse < best_mse **then**
            Update best_model, best_predictions, best_true_values, best_mse
        **end if**
    **end for**
    Compute average and standard deviation of mse and mae over all experiments:
    avg_mse $=$ mean$(mse_list)$, std_mse $=$ std$(mse_list)$
    avg_mae $=$ mean$(mae_list)$, std_mae $=$ std$(mae_list)$
    **return** best_model, best_predictions, best_true_values,
    avg_mse, std_mse, avg_mae, std_mae
**end function**

下面就不同模型分别进行训练与评估，由于不同模型的超参数类型不一致，难以进行严格的比对，只能控制其实验的次数、训练的轮数以及学习率保持一致。此外，题目要求短期预测和长期预测需要设置各自的超参数。详细参数设置可参考附录3和附录4，训练与评估的过程可参考项目中的 train_process 文件夹，完成训练后会分别绘制出局部预测和整体预测的样图，并给出三个模型的 MSE 与 MAE 的均值和标准差。

# 3 Results

## 3.1 Comparison and analysis

### 3.1.1 Comparison for short-time

| Model | MSE (Mean) | MSE (Std) | MAE (Mean) | MAE (Std) | Training Time |
|---|---|---|---|---|---|
| LSTM | 30886 | 2389 | 123 | 4.9 | 9 min |
| Transformer | 31273 | 1787 | 119 | 3.7 | 10 min |
| N-BEATS | 36560 | 8980 | 85 | 5.3 | 6 min |

Table 1: Comparison for short-time prediction

### 3.1.2 Comparison for long-time

| Model | MSE (Mean) | MSE (Std) | MAE (Mean) | MAE (Std) | Training Time |
|---|---|---|---|---|---|
| LSTM | 31538 | 2085 | 117 | 3.7 | 20 min |
| Transformer | 36334 | 1909 | 127 | 3.9 | 22 min |
| N-BEATS | 28465 | 3805 | 111 | 9.1 | 12 min |

Table 2: Comparison for long-time prediction

### 3.1.3 Analysis

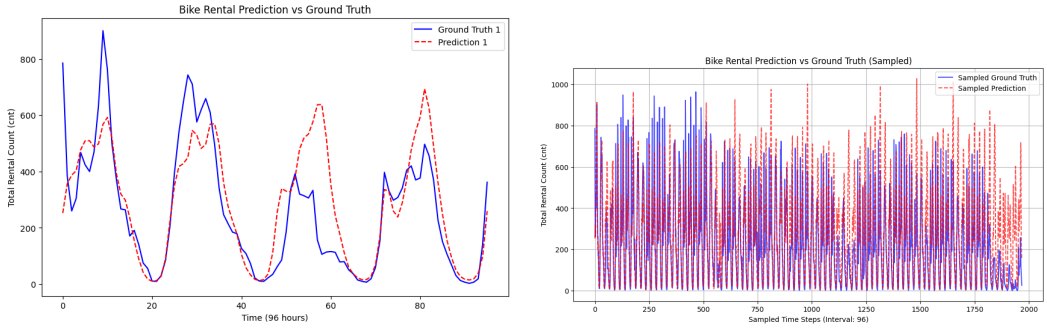
在短期预测任务中，Transformer以较低误差和出色稳定性表现最佳。LSTM 凭借最低 MSE 均值（30886）占优，但稳定性稍逊于 Transformer。N-BEATS 的 MAE 均值最低（85）且训练最快（6 分钟），效率突出，但稳定性较差（MSE 标准差 8980）。总体而言，LSTM 适合追求低误差场景，N-BEATS 适合对效率和绝对误差敏感的需求，Transformer 则在稳定性上表现优异，适合注重预测一致性的场景。

在长期预测任务中，N-BEATS以最低的 MSE（28465）和 MAE（111）均值，以及最短的训练时间（12 分钟）表现最佳，适合对误差和效率要求高的场景。LSTM 在误差标准差上表现更稳定，但均值略高于 N-BEATS。Transformer 表现均衡，稳定性最佳（最低 MSE 标准差 1909），但误差均值较高（MSE 36334，MAE 127）。总体而言，N-BEATS 是优选模型，LSTM 和 Transformer 适合注重稳定性的需求。

关于 N-BEATS 稳定性差的原因，可能是由于 N-BEATS 的深度和块结构较为复杂增加了训练的不确定性，或是时间序列数据的趋势性或季节性不明显，N-BEATS 的分解能力可能无法充分发挥，还可能是多特征拼接引入了噪声。

## 3.2 Short-time prediction

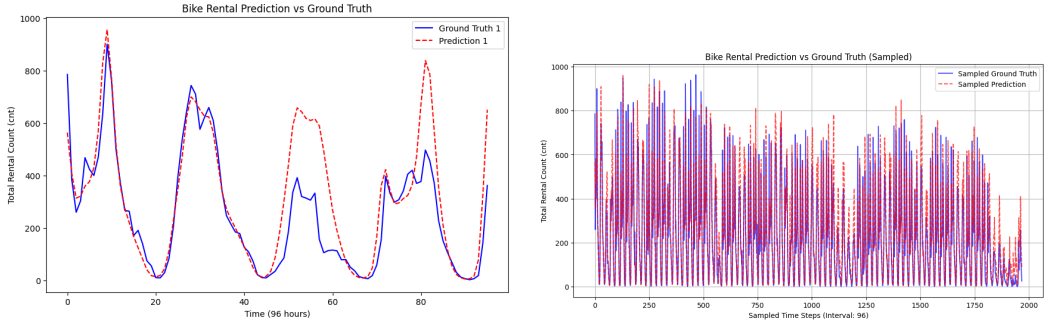
### 3.2.1 LSTM




(a) part (b) all

Figure 2: LSTM 短期预测

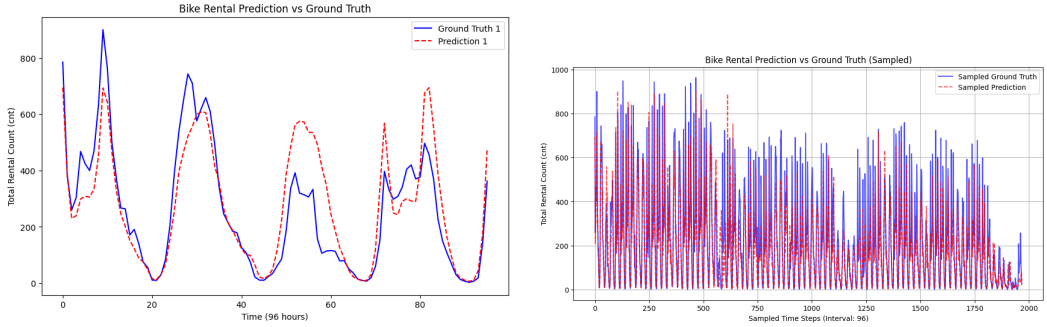
### 3.2.2 Transformer




(a) part (b) all

Figure 3: Transformer 短期预测

### 3.2.3 N-BEATS

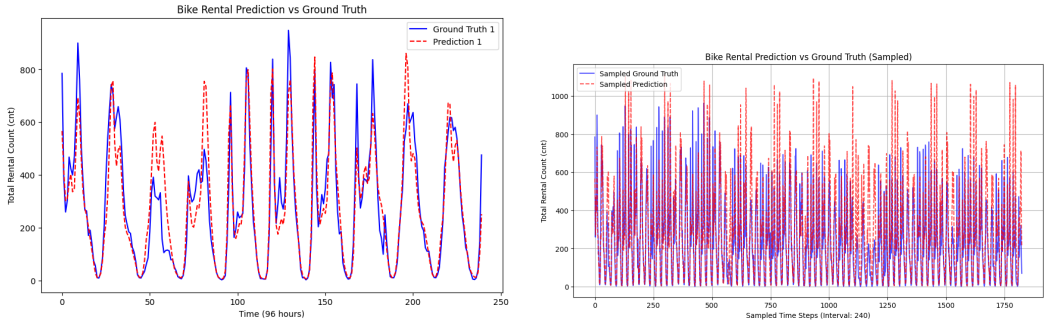



(a) part (b) all

Figure 4: N-BEATS 短期预测

## 3.3 Long-time prediction

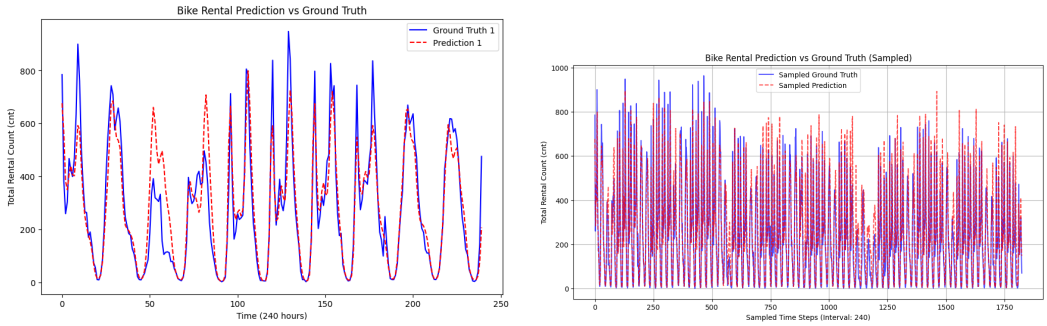
### 3.3.1 LSTM




(a) part (b) all

Figure 5: LSTM 长期预测

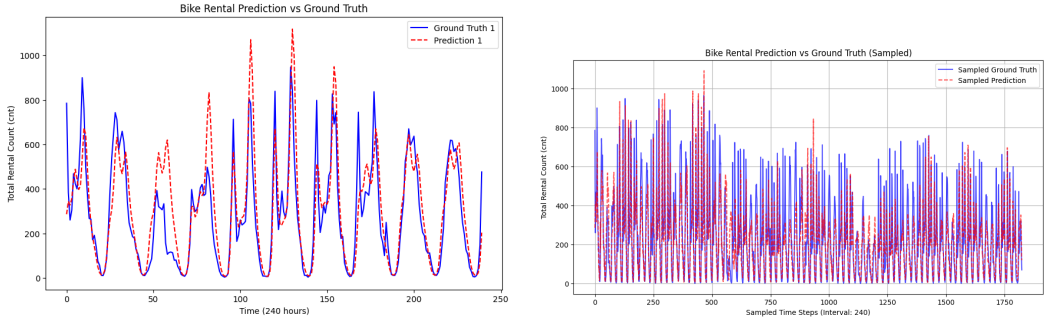
### 3.3.2 Transformer




(a) part (b) all

Figure 6: Transformer 长期预测

### 3.3.3 N-BEATS




(a) part (b) all

Figure 7: N-BEATS 长期预测

# 4 Discussion

以下讨论均与孙游阳通过腾讯会议交流得到，隶属于组间讨论部分。

## 4.1 Features selection

**Sun:** 我通过数据相关度分析可以从这 14 个特征中筛选出 9 个与 cnt 相关度比较高的特征，进而输入模型处理，这样可以简化模型的参数量。
**Cai:** 这是一个不错的想法，这种简化理论上可作用于 LSTM 和 N-BEATS 模型，至于 Transformer 模型，其自身的注意力机制可以自动对相关度高的特征进行筛选。

## 4.2 Logarithmic processing

**Sun:** 我通过目标值分布分析发现在训练过程中可以将目标值 cnt 对数化处理，这样一方面可以简化建模和预测过程，降低复杂度，另一方面可以使得训练过程中的损失函数值不至于过大。
**Cai:** 确实如此，我一开始没有对数化目标值时，训练过程中的损失函数值确实是一个比较惊人的数字，现在我在训练过程中将目标值对数化处理，以降低损失值，在评估过程中将目标值进行反对数化处理，以获得在原尺度下的 MSE 和 MAE。

# 5 References

## 参考文献


[1] https://www.overleaf.com/latex/templates/stockholm-university-report-template/jzgkxrbznnfv

[2] https://zhuanlan.zhihu.com/p/617338771

[3] https://zhuanlan.zhihu.com/p/470803137

[4] https://blog.csdn.net/xxt228/article/details/128754364

[5] http://arxiv.org/abs/1905.10437

# A Hyperparameters of short-time prediction

| Hyperparameter | Value |
|---|---|
| num_features | 14 |
| input_size | 96 |
| output_size | 96 |
| num_layers | 3 |
| hidden_dim | 128 (LSTM, Transformer) / 256 (N-BEATS) |
| num_experiments | 5 |
| epochs | 40 |
| lr | 0.001 |
| num_heads (Transformer only) | 8 |
| num_blocks (N-BEATS only) | 6 |
| block_type (N-BEATS only) | seasonality |

Table 3: Hyperparameters of short-time prediction

# B Hyperparameters of long-time prediction

| Hyperparameter | Value |
|---|---|
| num_features | 14 |
| input_size | 96 |
| output_size | 240 |
| num_layers | 3 |
| hidden_dim | 256 (LSTM) / 128 (Transformer, N-BEATS) |
| num_experiments | 5 |
| epochs | 80 |
| lr | 0.001 |
| num_heads (Transformer only) | 4 |
| num_blocks (N-BEATS only) | 3 |
| block_type (N-BEATS only) | seasonality |

Table 4: Hyperparameters of long-time prediction

# C Run result screenshot

```
Overall Results:
MSE: Mean = 30886.1418, Std = 2389.2646
MAE: Mean = 123.2601, Std = 4.9437

Logs saved to LSTM_short_prediction_process.txt.
```

(a) short-time

```
Overall Results:
MSE: Mean = 31537.9880, Std = 2085.6243
MAE: Mean = 117.3074, Std = 3.7747

Logs saved to LSTM_long_prediction_process.txt.
```

(b) long-time

Figure 8: LSTM results

```
Overall Results:
MSE: Mean = 31273.6366, Std = 1787.1693
MAE: Mean = 119.1985, Std = 3.7685

Logs saved to transformer_short_prediction_process.txt.
```

(a) short-time

```
Overall Results:
MSE: Mean = 36334.7928, Std = 1909.2195
MAE: Mean = 126.6183, Std = 3.9422

Logs saved to transformer_long_prediction_process.txt.
```

(b) long-time

Figure 9: Transformer results

```
Overall Results:
MSE: Mean = 36560.5553, Std = 8979.9483
MAE: Mean = 85.7916, Std = 5.33503

Logs saved to N-BEATS_short_prediction_process.txt.
```

(a) short-time

```
Overall Results:
MSE: Mean = 28465.6505, Std = 3805.9467
MAE: Mean = 111.7329, Std = 9.1799

Logs saved to N-BEATS_long_prediction_process.txt.
```

(b) long-time

Figure 10: N-BEATS results